



TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
NATIONAL COLLEGE OF ENGINEERING

A
MINOR PROJECT REPORT
ON
BASOBAS: A MOBILE RENTAL ACCOMMODATION FINDER

SUBMITTED BY:

NITESH PANT (NCE079BCT024)
RAJESH BHANDARI (NCE080BCT027)
SARWAGYA ACHARYA (NCE080BCT038)
UTSAV BHATTARAI (NCE080BCT048)

SUBMITTED TO:

DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING

August, 2026

Page of Approval

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
NATIONAL COLLEGE OF ENGINEERING
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

The undersigned certifies that they have read and recommended to the Institute of Engineering for acceptance of a project report entitled ‘**BasoBas**’ submitted by **Nitesh Pant, Rajesh Bhandari, Sarwagya Acharya, Utsav Bhattarai** in partial fulfillment of the requirements for the Bachelor’s degree in Electronics, Information and Communication Engineering & Computer Engineering.

.....

Supervisor

Subash Pandey

Designation

Department of Electronics and Computer

Engineering,

National College of Engineering, IOE, TU.

.....

External examiner

Full Name

Designation

Affiliation

Date of approval:

Copyright

The author has agreed that the Library, Department of Electronics and Computer Engineering, National College of Engineering, and Institute of Engineering may make this report freely available for inspection. Moreover, the author has agreed that permission for extensive copying of this project report for scholarly purposes may be granted by the supervisors who supervised the work recorded herein or, in their absence, by the Head of the Department wherein the project report was done. It is understood that recognition will be given to the author of this report and the Department of Electronics and Computer Engineering, National College of Engineering, IOE for any use of the material of this project report. Copying publication or the other use of this report for financial gain without the approval of to the Department of Electronics and Computer Engineering, National College of Engineering, Institute of Engineering, and the author's written permission is prohibited.

Request for permission to copy or to make any other use of the material in this report in whole or in part should be addressed to:

Head
Department of Electronics and Computer Engineering
National College of Engineering
Talchhikhel, Lalitpur.

Acknowledgments

On this page, the author expresses his gratitude and concern by using praising and thanksgiving words. don't forget to include your project supervisor's name. p.s. all the names must be bold.

Abstract

Abstract represents a summarized report of the complete project in a very concise and informative format covering the main objective and aim of the project, the background information, processes and methods used, and methodologies implemented, a brief conclusion of two to three lines talking about the results and scope of the project followed with 3 to 5 keywords (*italic*). The entire abstract of a project report should be written in about 250 to 350 words.

Keywords: *keyword 1, keyword 2, keyword 3, keyword 4, Keyword 5*

Contents

Page of Approval	ii
Copyright	iii
Acknowledgements	iv
Abstract	v
Contents	viii
List of Figures	ix
List of Tables	x
List of Abbreviations	xi
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Aim and Objectives	3
1.4 Scope	3
2 Literature Review	5
2.1 Related work	5
2.2 Related theory	8
2.2.1 Two-Sided Platforms and Matching	8
2.2.2 Relational Data Modelling	8
2.2.3 Geospatial Distance (Haversine Formula)	9
2.2.4 Rating Aggregation	10
2.2.5 Basic Security and Privacy	10
3 Methodology	11
3.1 Feasibility Study	11
3.1.1 Technical Feasibility	11
3.1.2 Operational Feasibility	11

3.1.3	Economic Feasibility	11
3.1.4	Schedule Feasibility	12
3.1.5	Legal/Ethical Feasibility	12
3.2	Implementation Details	13
3.2.1	Frontend / Mobile App Development	13
3.2.2	Backend / Database	13
3.3	Module Implementation	13
3.3.1	UI/UX Design	13
3.3.2	Authentication and Onboarding Module	14
3.3.3	Property Listing Management Module	14
3.3.4	Property Discovery Module	15
3.3.5	Visit Request Workflow Module	16
3.3.6	Landlord Experience Module	16
3.3.7	Tenant Experience Module	17
3.3.8	Pro Plan and Payment Module	17
3.3.9	Notification System Module	18
3.3.10	Ratings and Reviews Module	19
3.3.11	KYC Verification Module	19
3.3.12	AI Recommendation Module	20
3.3.13	Profile and Account Management Module	20
4	System Design	22
4.1	Requirement Specification	22
4.1.1	Functional Requirements	22
4.1.2	Non-Functional Requirements	24
4.1.3	System Architecture	24
4.1.4	Use Case Diagram	25
5	Results & Discussion	27
5.1	Software Development Approach	27
5.2	Testing, Evaluation and Validation	27
5.2.1	Test Environment	28
5.2.2	Unit Test Cases	28
5.2.3	Integration Test Cases	31
5.2.4	System Test Cases (End-to-End Journeys)	36
5.2.5	KYC Verification Evaluation	38
5.2.6	Security Test Cases	39

5.2.7	Traceability Summary	40
6	Conclusions	42
7	Limitations and Future enhancement	43
	References	43
	Appendices	46

List of Figures

3.1	Authentication and Onboarding Workflow	15
3.2	Visit Request Status Lifecycle	16
3.3	eSewa Payment Transaction Flow	18
3.4	Architecture of KYC Verification Process	20
4.1	System Architecture of BasoBas	24
4.2	Use Case Diagram for BasoBas	26

List of Tables

2.1	Core Relational Entities for BasoBas	9
3.1	Economic Feasibility Estimation	12
5.1	Test Environment	28
5.2	Unit Test Case UT-01 – Normalization of a Valid Mobile Number	29
5.3	Unit Test Case UT-02 – Rejection of Malformed Mobile Numbers	29
5.4	Unit Test Case UT-03 – Name Comparison for Identical Names	30
5.5	Unit Test Case UT-04 – Composite Risk Score Calculation	30
5.6	Unit Test Case UT-05 – Property Status Derivation Threshold	31
5.7	Integration Test Case IT-01 – Sign-In Creates a Profile Record	32
5.8	Integration Test Case IT-02 – Landlord Onboarding with KYC Submission .	33
5.9	Integration Test Case IT-03 – Listing Publication with Photos	34
5.10	Integration Test Case IT-04 – Visit Acceptance Unlocks Address Visibility .	35
5.11	Integration Test Case IT-05 – eSewa Payment Activates Pro Plan	36
5.12	System Journey SYS-J1 – Tenant Registration to Visit Request	37
5.13	System Journey SYS-J2 – Landlord Listing to Rental Finalization	37
5.14	System Journey SYS-J3 – eSewa Subscription Payment	37
5.15	KYC Evaluation Dataset Composition	38
5.16	Confusion Matrix for Tamper Detection	38
5.17	Security Test Case ST-01 – Location Hidden Without Approved Visit	39
5.18	Security Test Case ST-02 – Direct Access to Another User’s Document . . .	40
5.19	Security Test Case ST-03 – Cross-User Modification Blocked	40
5.20	Security Test Case ST-04 – OTP Brute-Force Lockout	40
5.21	Test Suite Traceability Summary	41

List of Abbreviations

LAH List of Abbreviations Here

AMC Add More Contents

1. Introduction

1.1 Background

Rental accommodation is a housing arrangement in which a property owner allows another person to live in a room or property in exchange for rent. It is one of the most common ways people find a place to live, especially students, working professionals, and families who move to a new area for education, work, or other reasons. Renting provides a practical option for people who need a place to stay without buying a house.

The rental process mainly involves two people: the landlord and the tenant. The landlord offers a room or property for rent by deciding details such as the rent, available facilities, availability, and tenant preferences. The tenant looks for a room that matches their budget, preferred location, and other needs. When both sides have complete and accurate information, it becomes easier for tenants to find a suitable place and for landlords to find suitable tenants.

In Nepal, finding a rental room is still largely an informal process. Every year, many students and young professionals move to the Kathmandu Valley from different parts of the country for higher education and employment, making the search for accommodation one of their first priorities [1]. Most people depend on recommendations from friends and relatives, social media groups, local advertisements, or real estate agents to find available rooms. Because information is scattered across different sources, tenants often spend a lot of time contacting landlords and visiting multiple properties before finding a suitable room. It is also common to discover that a room is no longer available or that the rent, facilities, or tenant preferences are different from what was initially advertised.

Nepal has online rental platforms such as KothaBhada and GharBheti that help landlords advertise available properties. These platforms have made rental listings more accessible, but they are designed to cover many types of properties rather than focusing on rental rooms. As a result, details that many tenants consider important, such as whether a room is suitable for students, bachelors, families, or working professionals, are not always available as searchable filters. In many cases, this information is only mentioned in the property description, making it harder for users to compare listings and find rooms that meet their requirements. Similar issues have been identified in studies of online rental platforms, where incomplete or unstructured listing information makes the search process less effective [2].

Location is another important factor when choosing a rental room. Students often pre-

fer rooms close to their college or university, while working professionals usually look for accommodation near their workplace. However, searching for nearby rooms is not always straightforward. Users often have to browse listings one by one instead of searching within a selected distance from a specific location.

Communication between landlords and tenants is also mostly handled through phone calls or messaging applications. After finding a suitable room, tenants need to contact the landlord to ask whether it is still available and arrange a visit. Landlords, on the other hand, have to manage every inquiry separately, especially when several people are interested in the same property. There is no simple way for either side to keep track of visit requests or their status.

To help improve this process, BasoBas is proposed as a mobile application for rental room discovery in the Kathmandu Valley. The application allows landlords to create property listings with clear information about the room, while tenants can search using filters such as location, budget, room type, facilities, and tenant preferences. It also includes features for nearby search, visit requests, KYC verification, and ratings and reviews.

The application is designed to keep the rental process simple. A tenant can browse available rooms, apply filters, view property details, and request a visit. The landlord can review the request and choose to accept, reschedule, or decline it. After visiting the property, tenants can leave ratings and reviews, helping future users make better-informed decisions.

1.2 Problem Statement

Tenants relocating to cities such as Kathmandu for study or work face significant difficulties in finding suitable rental accommodation. The discovery process remains informal and broker-dependent, while listing information is often incomplete or unstructured. Landlords may prefer specific tenant categories, but these rules are not consistently transparent on existing portals. This leads to mismatch, repeated travel, wasted time, and avoidable costs.

The major problem points are as follows:

1. Heavy dependence on brokers and informal contacts, with high commission and limited accountability.
2. Lack of clear, filterable tenant-acceptance criteria such as students allowed and bachelors allowed.
3. Weak support for proximity-based search around current location or selected reference points.
4. Limited workflow support for booking/visit requests and transparent status updates.

Therefore, a focused location-based mobile platform is needed to support structured matching between tenants and landlords.

1.3 Aim and Objectives

The aim of this project is to develop BasoBas Mobile Application that makes it easier for people to find rental rooms in the Kathmandu Valley. The application provides a single platform where tenants can search for rooms and landlords can list their properties, making the rental process simpler and more organized for both.

The objectives of this project are as follows:

1. To enable landlords to add and manage room listings by providing details such as rent, facilities, address, location, tenant preferences, and availability.
2. To help tenants find suitable rental rooms using map-based nearby search and filters such as location, budget, room type, facilities, tenant preferences, availability, and ratings.
3. To facilitate secure and efficient rental interactions through KYC verification for landlords and tenants, along with a visit request system that supports request, acceptance, rescheduling, and rejection.
4. To improve transparency and user confidence by incorporating a rating and review system that allows tenants to share their rental experiences and assist others in making informed decisions.

1.4 Scope

The scope of BasoBas is limited to providing a mobile platform for rental room discovery and property listing within Kathmandu Valley. The system supports two primary user roles: landlords and tenants, along with an administrator responsible for managing the platform. Landlords can publish and manage rental property information, while tenants can browse available listings, search for properties using various filters, view nearby rentals on a map, and submit visit requests. The application also includes KYC verification for user identity validation and a rating and review mechanism to improve trust and transparency among users.

The application is developed for Android using React Native with Supabase as the back-end service for authentication, database management, and file storage. The system stores user profiles, property information, visit requests, reviews, and verification records in a centralized PostgreSQL database. The project focuses only on facilitating the rental property

search and listing process and excludes features such as online rent payment, digital rental agreements, dispute resolution, and other property management services beyond the scope of the prototype.

2. Literature Review

2.1 Related work

To better understand what has already been done in the area of online room-finding and rental platforms, a number of related research papers were read and reviewed. Each paper was studied to understand what problem they were trying to solve and how they built their solution. The results they achieved and what was still missing in their work was also noted. This helped in identifying the gaps that the present work aims to fill. A summary of each reviewed paper is presented below.

Indrayani and Istikomah created a web-based platform for boarding-house rental services in Pati Regency, Indonesia, focusing on information architecture and booking workflow [2]. The system followed the Waterfall development model, with requirements gathered through semi-structured interviews and field observations. Then, through the use of UML, the use case and class diagrams were modeled. At last, black-box testing was conducted for object validation. No large external dataset was used. Instead, the data consisted of qualitative interview and observation records used to derive functional and non-functional requirements. The results showed that all tested modules passed black-box testing without critical defects, confirming the functional reliability of the system. A clear limitation is that the system depends on manual payment confirmation rather than an automated payment gateway.

Diaz et al. developed RoomMeet, a rental mobile application that targets a variety of users, such as students, families, and working professionals [3]. The platform was developed using an Extreme Programming (XP) methodology, where user stories were gathered through surveys guided iterative low-fidelity prototyping. Data for the study came from a 50-respondent survey, of which 74% had previously searched for rooms or apartments to rent, along with System Usability Scale (SUS)-style ratings of the prototype. Results showed that users highly accepted the idea, with most respondents rating the prototype as better than existing rental search alternatives and identifying search filters as the most useful feature. Yet, there was no actual deployment and testing on the large scale, since the evaluation was limited to a survey-based prototype rather than a functioning system with live landlords and tenants.

Farooq et al. proposed RoomMate, an AI-optimized space rental system that is mainly intended for student housing and room-sharing purposes [4]. The system was built using an Agile Software Development Life Cycle and supplemented by a Dialogflow-based AI chatbot

for natural-language room search and FAQ handling. No empirical dataset is mentioned. Instead, they used requirement gathering through interviews and use-case/user-story development to define system functionality. The final product was a role-based platform offering verified listings, roommate matching, and chatbot-assisted search, which was validated through unit integration end-to-end, and performance testing. Still, the paper does not report any live-user evaluation, which means that the actual accuracy of the AI chatbot and the platform’s real-world trust-building claims remain unverified beyond internal testing.

Hess et al. examined neighborhood representation on internet rental housing platforms [5]. In their study, they compared Craigslist and Apartments.com about space, platform type, and metropolitan segregation. The researchers used web-scraping to collect the listings with the help of Helena and Python scripts. The dataset covered 847,068 Craigslist listings and 462,073 Apartments.com listings across the 100 most populated U.S. metropolitan areas from January to August 2019. The results revealed that high-income, highly educated, and low-poverty neighborhoods are consistently overrepresented on both platforms, while Black and Latinx neighborhoods are systematically underrepresented on Craigslist in more segregated metropolitan areas. One limitation of the study is that it only focuses on two platforms and also does not capture real-time changes in actual rental availability.

Zaman et al. presented an interactive web platform and Android application that facilitate a room-sharing scheme for students in Bangladesh facing accommodation shortages during admission-test season [6]. The system was built using PHP for the website and a corresponding Android application, with a layered moderation structure of global administrators and area-specific local moderators, and was evaluated through load testing and a user survey. Load-testing data showed that 50 virtual users produced 10,803 requests within five minutes of testing. Results indicated suggesting general user satisfaction with the prototype. The work, however, lacked working payment or fully automated verification mechanisms. They recognized the interface was not fully user-friendly and the platform was limited to the Android operating system only.

Tyagi et al. proposed RoomRover, a smart room rental platform that uses conversational AI for helping users book rooms [7]. The platform was built on the MERN stack, integrating an AI-powered chatbot. This chatbot was created through different stages like gathering the requirements, designing the system, implementing it, and testing it. The study does not cite an external dataset but it shows the performance metrics of the system that was obtained after deploying the chatbot. The findings showed that the chatbot was responding under 3 seconds, the system was available 99.9% of the time, 85% of users’ requests were resolved without human assistance, and 30% due to personalized recommendations. However, that the system lacks wider adoption testing, and reliance on AI chatbots may not be universally

accepted by users who prefer human assistance.

Balpande et al. developed a web-based room-searching platform called Room Dhundo, targeting tenants and room owners in India [8]. The core objective of their work was to eliminate broker dependency and provide a centralized, affordable, and transparent accommodation-finding system. To achieve this, they built the platform using Python for the backend, enhanced it with Google Geolocation APIs, a real-time chat module, and an AI-powered chatbot to handle frequently asked questions. The platform also integrated a mess service finder and a carpooling feature to address students' broader daily needs. Results demonstrated that the system successfully connected tenants and landlords in a more efficient and transparent manner, reducing communication delays through the built-in chat feature. However, the work doesn't include any formal user evaluation or performance benchmarking.

Wakodikar et al. presented the design and implementation of a web-based To-Let (Room Finder) application intended to automate and streamline rental property management in urban India [9]. Their main reason was to eliminate the manual processes involving brokers that were not only slow but were also very prone to errors. The methodology followed a structured sequential approach supporting full CRUD (Create, Read, Update, Delete) operations. A comparative analysis presented in the paper showed that the proposed system outperformed manual methods and existing partial-digital systems in terms of data organization, automation, processing speed, and scalability. However, the application remains a basic prototype; it does not include advanced features such as search filters by preference, geolocation-based discovery, user reviews, or a communication channel between tenants and landlords, which significantly limits its practical usability in a real-world urban rental scenario.

Fensen, Dewayani, and Wasino designed a website application specifically for locating and reserving houses and boarding rooms for rent, targeting students enrolled in universities in Jakarta, Indonesia [10]. They identified the increasing demand for housing near universities as the biggest reason behind the rising rental prices. The data were gathered through semi-structured interviews, focus group discussions with students, and surveying, whereas primary data consisted of photographs of real properties. A notable technical feature was the implementation of the Haversine formula to calculate the distance of a listed property from the user's current or selected location, enabling search results filtered by radius and administrative region. The application also supported an online booking module, allowing users to reserve rooms directly and track their rental history. Despite these strengths, the system was limited to a specific geographic area around one university campus, and the study did not report any quantitative evaluation of user satisfaction or system accuracy, leaving questions about scalability and broader adoption unanswered.

Okechi John Uchechukwu and Aisha Abdulrazaq Shehu highlighted an often-neglected aspect of housing, i.e. the problems experienced by foreign migrants, mainly Africans, when looking for rental housing in Russia [11]. Their work introduced Afro-Kiva (later rebranded as Miradom), a culturally aware, multilingual digital platform designed to reduce language barriers, counter discriminatory practices by landlords, and provide a trusted, verified rental experience. The methodology was comprehensive, combining literature review, online surveys of African students and expatriates, interviews with real estate agents, focus group discussions, and competitor analysis of existing Russian platforms such as Avito and Sutochno. Quantitative findings from their surveys revealed that over 60% of African students experienced serious difficulties in finding culturally inclusive housing. The platform was developed following user-centered design principles and proposed features including verified property listings, multilingual support, a dedicated customer support center, and value-added services such as noise monitoring and cleaning. However, it was just a preprint and prototype stage; the platform had not been deployed or tested at scale at the time of publication.

The studies we’ve looked at together reveal that digital rental platforms have drastically changed the way we find accommodation. Instead of relying on informal and broker-dependent methods, these platforms offer structured online listing, searching, and booking features. Such attributes as mobile-based approach, filters for searching, artificial intelligence algorithms of recommendations, and trust factors including rating and certified listing have been proved to increase users’ satisfaction and decrease search friction. At the same time, the reviewed literature does not cover any issues of the Nepali rental market, such as tenant screening, proximity-based discovery, and visit scheduling. BasoBas is planned to be the solution to these problems.

2.2 Related theory

2.2.1 Two-Sided Platforms and Matching

BasoBas functions as a two-sided digital platform that connects two user groups: tenants and landlords [12]. The platform’s value depends on efficient matching between tenant preferences and landlord constraints. Tenants typically optimize for rent, location, room type, and acceptance policy, while landlords optimize for appropriate occupant profile and reliable tenancy. Structured listing attributes convert these preferences into filterable parameters and improve match quality.

2.2.2 Relational Data Modelling

The core BasoBas entities are well suited to relational representation. Primary keys uniquely identify records, while foreign keys enforce relationships and referential integrity. Normalized

tables reduce redundancy and support reliable updates [13].

Table 2.1: Core Relational Entities for BasoBas

Entity	Primary Key	Important Attributes / Foreign Keys
Profiles	clerk_id	phone, full_name, active_role, city
LandlordProfile	clerk_id (FK)	verification_status, avg_rating, total_reviews, is_phone_shared_default
KYCSubmission	id	clerk_id (FK), document_type, status, submitted_at, reviewed_at
Listing	id	landlord_clerk_id (FK), title, description, property_type, status, price, address, city, locality, latitude, longitude, bedrooms, bathrooms, furnished_status, parking, amenities, available_from
ListingImage	id	listing_id (FK), image_path, display_order, is_cover
VisitRequest	id	listing_id (FK), tenant_clerk_id (FK), landlord_clerk_id (FK), requested_date, requested_time_slot, status
SavedListing	id	tenant_clerk_id (FK), listing_id (FK), saved_at
Product	id	name, price, duration_months, active
Transaction	id	clerk_id (FK), product_id (FK), total_amount, status, esewa_ref_id
UserPass	id	clerk_id (FK), product_id (FK), transaction_id (FK), starts_at, expires_at, status

2.2.3 Geospatial Distance (Haversine Formula)

For nearby-room functionality, BasoBas computes approximate surface distance between user/reference coordinates and property coordinates using the Haversine formula [14]. Let user coordinates be (ϕ_1, λ_1) and property coordinates be (ϕ_2, λ_2) in radians:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1) \cos(\phi_2) \sin^2\left(\frac{\Delta\lambda}{2}\right) \quad (2.1)$$

$$c = 2 \arctan 2\left(\sqrt{a}, \sqrt{1-a}\right), \quad d = R c \quad (2.2)$$

where $\Delta\phi = \phi_2 - \phi_1$, $\Delta\lambda = \lambda_2 - \lambda_1$, and $R \approx 6371$ km. A property is included in nearby results if $d \leq r$, where r is the selected radius.

2.2.4 Rating Aggregation

A basic reputation layer can be implemented through aggregated tenant ratings. If a property has ratings $r_1, r_2, \dots, r_n$, the average rating is computed as:

$$\bar{R} = \frac{1}{n} \sum_{i=1}^n r_i \quad (2.3)$$

This aggregate score helps users rank and filter listings and aligns with standard on-line marketplace reputation design principles [15]. Related feedback-system analysis also supports this approach [16].

2.2.5 Basic Security and Privacy

Security and privacy principles considered in BasoBas include:

- Authentication and authorization so users can only perform allowed actions.
- Secure password handling in hashed form if backend authentication is used.
- Explicit location permission before enabling nearby search mode.
- Limited location usage for search and navigation only, without continuous background tracking.
- Manual area-search fallback when GPS permission is denied.

These controls reflect known LBS privacy considerations discussed in prior work [17]. Additional legal discussion is provided in [18].

3. Methodology

This chapter should contain the methodology implemented for problem-solving. It should contain all the work done from feasibility study, requirement analysis and data collection, system design to testing and implementation.

3.1 Feasibility Study

3.1.1 Technical Feasibility

The project was found to be technically feasible with available tools and institutional resources.

- Frontend: React Native (Expo) was used as the cross-platform mobile development toolkit.
- Backend: Supabase with PostgreSQL provided the managed database, authentication, realtime updates, and Edge Functions used across all modules.
- Map Integration: Google Maps SDK was integrated for map rendering, nearby-room visualization, and navigation support.
- Development and collaboration tools: VS Code was used for development and Git/GitHub supported version control and team collaboration throughout the project.

3.1.2 Operational Feasibility

The system was confirmed to be operationally feasible because target users already use smartphones for communication and property search activities. BasoBas formalizes existing behaviors into a structured workflow with low learning overhead for both tenants and landlords.

3.1.3 Economic Feasibility

The project was completed economically because core services remained available in free tiers, with only a small one-time fee required for Google Play deployment.

Table 3.1: Economic Feasibility Estimation

Cost Item	Estimated Cost
Supabase (DB, Auth, Realtime, Edge Functions, Hosting)	Free for minimal and prototype usage
Google Maps SDK	Free tier (up to usage limits)
Phone Number OTP Verification (SMS Charges)	Free test numbers + SMS credits; no additional cost.
Google Play Console (App Deployment)	US\$25 (one-time registration fee)
Development Tools (VS Code, Git, GitHub)	Free (open source / free plans)
Miscellaneous	Rs. 1,000–2,000
Total Initial Cost	Rs. 1,000–2,000 + US\$25 + OTP SMS usage charges

3.1.4 Schedule Feasibility

The project was scheduled to take 14 weeks, divided into 10 development stages: Requirement Analysis and Literature Review, System Design and Diagrams, React Native (Expo) Setup and Authentication, Supabase Schema, Storage and Row-Level Security (RLS), Search, Maps and Radius Filtering, Booking, Reviews and Realtime Features, System Integration and Refinement, Testing and Bug Fixing, and Final Report and Presentation Documentation.

During this period, 48 screens of the application were designed and developed. Lastly, all the features of the BasoBas mobile application have been implemented successfully. This confirms that the project was completed within the set schedule.

3.1.5 Legal/Ethical Feasibility

The completed prototype remained feasible under basic ethical and legal considerations:

- User data was collected only for platform functionality and demonstration.
- The location access is permission-based and can be controlled by the users themselves with a two-tiered privacy approach where the location information can be shared only upon acceptance of the visit request.
- No continuous surveillance logic was implemented.
- Data access control was implemented using Row Level Security (RLS) policies.

3.2 Implementation Details

3.2.1 Frontend / Mobile App Development

The mobile frontend was implemented using React Native with Expo and deployed through Expo’s managed workflow. This provided a single codebase for Android-first delivery with future iOS support. The implementation covers role-based tenant and landlord flows, including authentication, search and filtering, property details, booking requests, dashboard management, and review submission. Navigation will handle authenticated and unauthenticated routes, while reusable UI components will be used for listing cards, filter badges, and request status views. Expo modules were used for location permission and map redirection, and media upload support was implemented for property image handling. The deployment pipeline used Expo development builds and production Android package generation.

3.2.2 Backend / Database

The backend used Supabase as a cloud-hosted system from the initial deployment. Supabase PostgreSQL stores users, properties, booking requests, and reviews using foreign keys and indexed fields for filtering and scalable query performance. Supabase Auth manages secure registration and login with role-based access for tenants and landlords, and Row Level Security (RLS) policies enforce record-level authorization. Supabase Realtime provides live booking-status updates, Supabase Storage handles property images, and Edge Functions implement server-side validation, business rules, and future notification workflows.

3.3 Module Implementation

3.3.1 UI/UX Design

The entire interface of the BasoBas application is designed in Figma, covering all 48 screens of the platform. The visual language is minimalistic and built on a white primary color that conveys trust, with forest green as the sole brand accent to symbolize stability and growth. For a premium feel, headings use DM Serif Display while interface text uses DM Sans, and generous whitespace keeps every screen fresh and uncluttered. The onboarding journey is covered by seven screens, guiding the user from authentication to full profile setup: the landing screen, phone number submission, OTP verification, account creation, profile details setup, and identity verification. The tenant experience includes the home screen with an AI-recommendation section, search and filter interface, a large map view with property pins, property details, visit scheduling and tracking, notifications, saved properties, and profile management. Similarly, the landlord experience spans the dashboard with quick stats,

property management screens, a four-step listing creation flow, visit request management, and comparison screens. A complete Basic-to-Pro plan upgrade flow was also designed, comprising the features comparison screen with the payment option, payment receipt and failure screens, and locked feature indicators throughout the app. A defining element of the interface is the floating dock navigation: instead of a standard full-width bottom nav bar, the app uses a floating black frosted-glass pill that hovers above the screen content. This gives the app a luxurious and contemporary look while keeping the interface clean and visually uncluttered.

3.3.2 Authentication and Onboarding Module

Authentication starts when the user launches the application and enters their phone number, with Nepal being (+977) the default country code. Clerk, a third party identity provider is used here which sends a 6-digit One-Time Password (OTP) via SMS. The user enters the OTP, and Clerk authenticates it. Clerk was chosen because it provides out-of-the-box sign-in UI and takes care of session management, thus there is no need to implement authentication from scratch. After validating the phone number, Clerk generates a session token (JWT), which gets sent to Supabase. Supabase validates the token in every incoming request and utilizes Row-Level Security (RLS) policy in order to allow users to access only his data. If the user is logging in for the first time, the system validates the `onboarding_complete` field in his profile and if it is false, prompts the user through a quick onboarding flow. After that, they choose their role (Tenant or Landlord) by clicking on the appropriate card instead of selecting from the dropdown list. Then user enter their name and select the city, and in case of tenants, they choose property preferences. After completing this onboarding process, the value of `onboarding_complete` is set to true, and the user is redirected to their respective homepage. The data captured during this process is saved in three different tables: `profiles`, `user_roles`, and `user_preferences`.

3.3.3 Property Listing Management Module

Listings are generated by landlords using a structured multi-stage form rather than a single-page form. First, the landlord picks the type of the property as Apartment, House, Room, or Studio, and this determines what fields are available in the next step, because certain fields like number of bedrooms or floors apply only to Apartments, whereas certain tenant preference fields like gender restriction, student-friendly, or couple allowed apply to Rooms. Then, the landlord provides the pricing information, including the price per month, the negotiability of that price, and the earliest possible move-in date, after which he/she provides the location information in terms of address, city, locality, and optionally the latitude/longitude

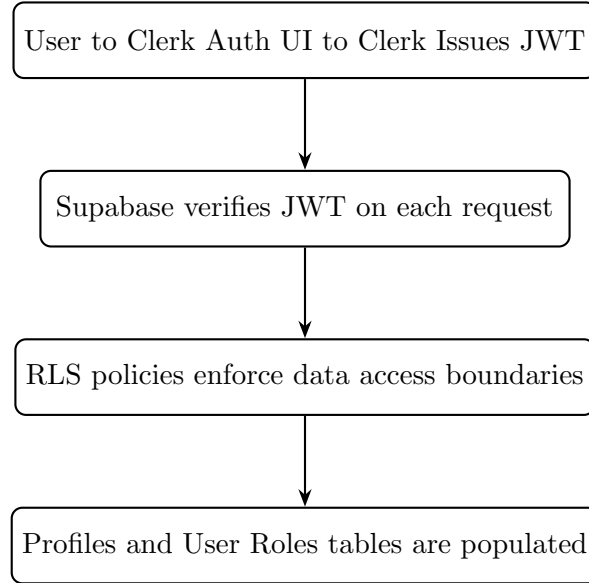


Figure 3.1: Authentication and Onboarding Workflow

of the property so that it appears on the map view. After that, he/she selects certain amenities out of a predefined list like WiFi, parking, and water supply, writes a brief description, and uploads multiple images into Supabase Storage with one selected as the cover photo. In total, there are six steps and throughout them, form submission is kept in a single state manager built around React Context so that nothing is erased if the user decides to change their mind about a certain step and goes back to it later.

3.3.4 Property Discovery Module

Tenants have three different options to look for listings: The first one is the home feed which is the most popular listings displayed based on a combination of recent activity, tenant preferences, and recommendations from the AI engine. Also, the tenants are being provided with a search and filter option where the tenants can filter search results for keyword, property type, price range, property size by minimum bedrooms, location, property-status(furnished/unfurnished) and move-in date. each filter acts as a query clause against the listings data. Filtering conditions are matched as query conditions to the listings table. In the map view, listings that have coordinate information are shown as pins on a map. Tapping a pin opens a card with a full listing link. In any of these views, when a listing is opened, it redirects to the detail page showing the complete image gallery and property details, landlord rating, and visit-request button.

3.3.5 Visit Request Workflow Module

Visit request procedure begins from the listing detail page, where the tenant chooses the date and time and makes the request, creating the request in the status Pending. The landlord, in turn, receives the request on his dashboard and accepts it, rejects with the option of specifying the reason, or offers a new date and time as part of the reschedule. Thus, returning the request to the status Pending to give the response to the tenant. Having accepted the request, the landlord has the opportunity to send two more types of information. They are the precise pin location of the property and a contact phone number, but those stay hidden until acceptance as a part of protecting the privacy of the landlord by default. When the visit has been completed, the request can be changed into the status Completed and allows the tenant to write a review, whereas the tenant can cancel the request at any moment prior to its completion.

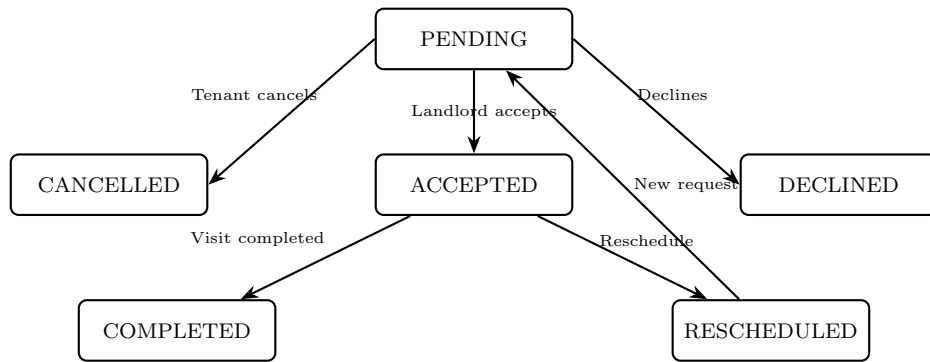


Figure 3.2: Visit Request Status Lifecycle

3.3.6 Landlord Experience Module

The landlord module of the app uses a four-tab bottom navigation bar with options as Dashboard, Requests, Alerts, and Profile. The Dashboard is the primary landing screen for the landlord and has card-based listings for active listings, pending requests, and KYC verification status. In addition, there is a listing of the properties belonging to the landlord with an option to edit, deactivate, or declare that the property is being rented, along with previews of the visit request received for that particular property. By clicking on any particular property from this listing, the user accesses all the relevant information associated with that listing, including photo galleries and visit request history. This tab is used to show all the visit requests according to their status such as Pending, Upcoming, or Past.

3.3.7 Tenant Experience Module

Upon signing in, the user enters their personal home feed that is created based on the preferences of the user and the recommendation signals, backed up by the fast filters and a search field. The separate Requests tab displays all the requests of the user sorted chronologically along with the status of the request, and once the landlord approves the request and sends location and contact details to the user, the latter is informed about it and able to view the information in the request field. There is also the Saved Listings tab for the tenants that allows saving the properties of interest.

3.3.8 Pro Plan and Payment Module

This module processes the subscription payments that the landlords require to make their listings active. Since, eSewa is the most popular digital wallet in Nepal, we use it as the payment gateway here. The subscription plans along with their cost and period of validity are stored in the form of a table named products so that they could be modified in the future without changing the code. Once a landlord selects a plan, the application creates a new transaction whose status is Pending for now and sends him/her to the eSewa payment portal. After making payment through eSewa, the user is redirected back to BasoBas where the Supabase Edge Function verifies this payment by making an API call to the eSewa server. If verification succeeds, then the transaction's status changes to Complete and a new user_passes entry is made with the start and expiry dates, Otherwise, the transaction's status changes to Failed. To prevent being charged twice due to problems in the network, every transaction is processed with the use of the idempotency key, while the webhook listener serves as the official authority on the matter as to whether the transaction was successful or not. Prior to activating the listing, the system confirms that the landlord has a valid pass and not expired one.

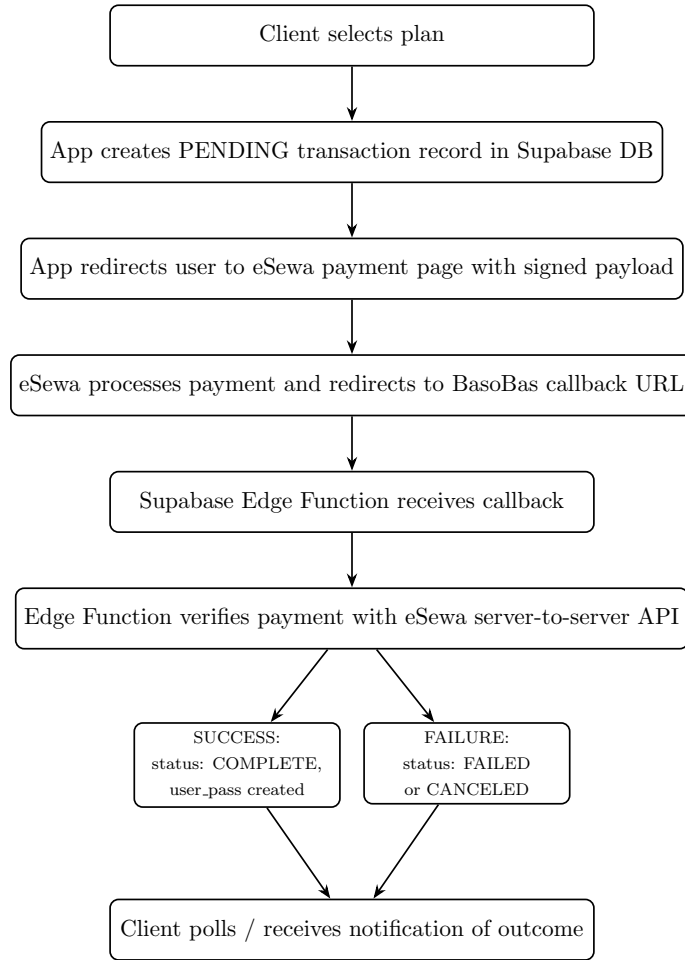


Figure 3.3: eSewa Payment Transaction Flow

3.3.9 Notification System Module

A set of trigger events was also outlined beforehand, which include such triggers as a new request to visit, request acceptance/rejection/rescheduling, sharing the location/contact, KYC being changed, payment being completed, or expiration of a pass. Each time any of these triggers takes place, a notification object is added to the notifications table, and Supabase Realtime is utilized to make sure that the notifications are available instantly on the user's device without the need for constant checks from the application itself. In case of notifications that have to be delivered to the user even when the application is closed, the use of Supabase Edge Functions results in delivering push notifications via the push notification service on the device. The navigation menu of tenants/landlords contains an Alerts tab where the history of notifications is shown along with unread/read status.

3.3.10 Ratings and Reviews Module

Tenants can provide a review only if his or her visit request status is completed, in order to prevent any faked reviews by those tenants who have not even seen the property. Upon the submission of the review, the landlord's average rating is updated according to the running-average algorithm, and thus there is no need for recalculating the rating from scratch every time, as well as the whole process is made through one database transaction in order not to allow two concurrent reviews to corrupt the average calculation. The final rating along with the review count are displayed as a star rating. The aggregate rating is computed as a running average using the formula:

$$\text{new_avg} = \frac{(\text{old_avg} \times \text{total_reviews}) + \text{new_rating}}{\text{total_reviews} + 1} \quad (3.1)$$

3.3.11 KYC Verification Module

The KYC process involves automation via AI and human-in-the-loop in case of ambiguities. The process starts with selecting the document type (Citizenship Certificate or National ID), uploading images of both sides of the document and optionally an electricity bill, which acts as additional proof of address and generates a submission with status 'Under Review'. The documents are uploaded to a private and controlled access bucket in Supabase Storage service and the paths to the file are added to the database. Next, a Supabase Edge Function runs the AI verification pipeline which involves reading the text from the document using OCR, verifying the document for any signs of forgery, comparing the document image to the selfie, checking consistency of the extracted information and returns a confidence score from 0 to 1. When the score is high enough, the submission gets status 'Verified' or 'Rejected', while lower score results in flagging and sending the submission to the Admin Portal. The administrator then reviews the documents, AI conclusions and the confidence score before making the final decision. If the submission is rejected, the administrator can also provide the reason for the rejection. This decision makes changes to both the submission history and the landlord's profile for consistency throughout the app. The landlord is informed about their decision, and if it is rejected, they have an opportunity to resubmit corrected information. This process continues until all attempts are made by the landlord.

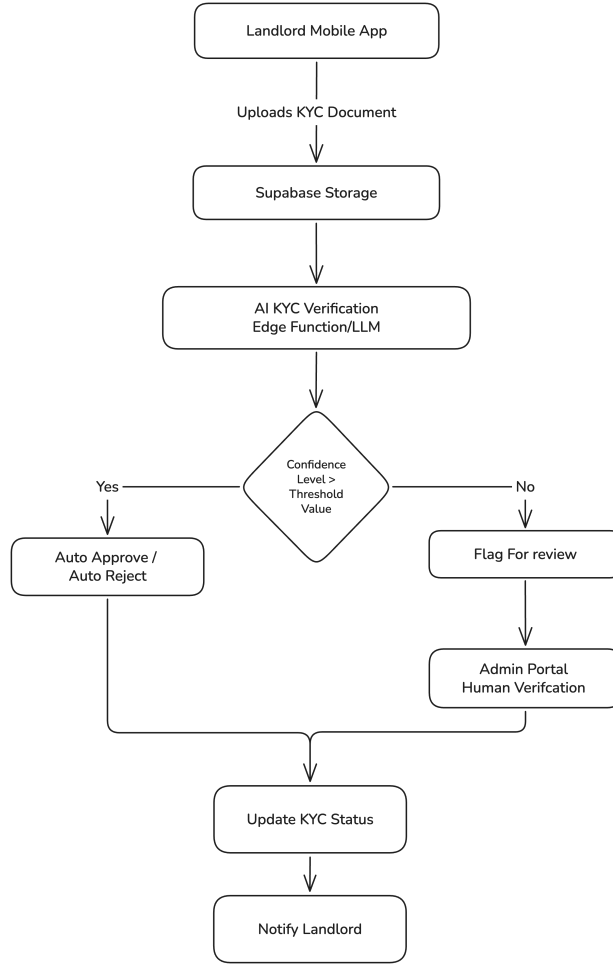


Figure 3.4: Architecture of KYC Verification Process

3.3.12 AI Recommendation Module

Recommendation engine functions based on explicit feedback from tenants such as their preferences for certain properties, saved searches and visit requests. In addition to that, it analyzes listing signals such as property type, price, amenities, location, landlord's score which are considered as quality signals. All the above signals are analyzed using a hybrid ranking system where the listings that are similar to what tenant likes before get ranked higher using content based filtering, and the listings that are popular among other tenants with similar preferences get ranked higher using collaborative filtering.

3.3.13 Profile and Account Management Module

On the Profile Screen, there is a profile photo of the user, name, phone number, and city, but the Clerk ID underneath is not changeable. Photos uploaded on the avatar are saved in Supabase Storage and follow a uniform naming convention such that the older photo will be replaced easily by a new one uploaded. A user who is registered to both tenant and

landlord roles can toggle between roles from the profile screen, and the application renders the appropriate navigation and home screen without any need for logout. For those who have logged in as the landlord role, the profile screen also has a KYC Status panel, Active-pass panel containing information about the plan currently being used, and the expiry date, and Rating information. The Settings screen completes this module with a toggle button to automatically share the phone number after accepting the visit request, notifications options, and account deletion process.

4. System Design

This chapter should include Requirement Specification (Functional and Non- Functional requirement) and all the diagrams that contribute to best graphically representing the system, its components & operations. They may be Context, Interaction, Structural, Behavioral models, network diagrams, model architecture or circuit diagrams etc.

4.1 Requirement Specification

4.1.1 Functional Requirements

FR1: User Registration and Authentication

- Users can register as tenant or landlord.
- The system supports phone number OTP verification during authentication flow.
- Registered users can log in and maintain session state securely.

FR2: Landlord Listing Management

- Landlords can create, update, and delete rental listings.
- Listings include room/flat type, rent, facilities, location coordinates, student acceptance flag, and availability.
- Landlords can attach listing images.
- Landlords can upload a short 10 to 15 second room-tour video for clearer property preview.

FR3: Tenant Search and Filtering

- Tenants can search listings by area/locality.
- Tenants can filter by rent range, room type, rating, and tenant-friendliness criteria.
- Results can be sorted by rent, distance, or rating.

FR4: Location-Based Nearby Discovery

- The system requests location permission when nearby mode is selected.

- Rentals within selected radius are identified using Haversine distance.
- Users can also search around a manually selected reference point.
- Matching rentals are displayed inside an embedded Google Maps SDK view with map markers and location-based browsing support.
- Rental details can still provide “Open in Google Maps” option for external navigation.

FR5: Booking/Visit Request Handling

- Tenants send request with desired move-in date and optional note.
- Landlords can accept, reject and reschedule requests.
- Landlords can share property location and contact number after accepting the request.
- Tenants can track request status (pending/accepted/rejected).

FR6: Ratings and Reviews

- Tenants can submit ratings and short reviews.
- Average rating is computed and displayed on property pages.

FR7: Notification System

- The system sends user-facing notifications for OTP verification, booking-request submission, and booking-status updates.
- Tenants receive alerts when landlords accept or reject requests.
- Landlords receive alerts when new booking or visit requests are submitted.

FR8: AI KYC Verification

- Users can submit KYC documents (Citizenship Certificate or National ID along with Electricity Bill) for verification.
- The system uses AI to verify documents and flag ambiguous cases for human review.
- Users are notified of verification results and can resubmit if rejected.

4.1.2 Non-Functional Requirements

- **Usability:** Interface should be simple, clear, and easily navigable with minimal learning effort.
- **Performance:** Search, filtering, and detail loading should execute within acceptable time for prototype data volume.
- **Reliability:** Common workflows should be stable; failures should show meaningful error messages.
- **Security:** Access controls must prevent unauthorized data modification.
- **Privacy:** Location data usage must remain consent-based and purpose-limited.
- **Maintainability:** Modular structure should support feature extension and codebase evolution.

4.1.3 System Architecture

BasoBas follows a multi-layer client-server architecture with clearly separated presentation, administration, application, and data layers.

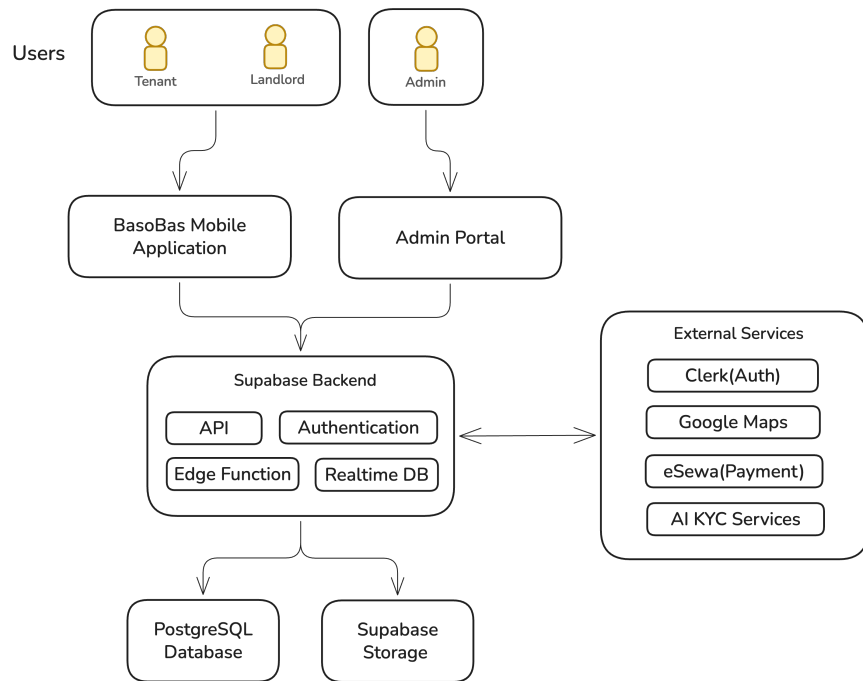


Figure 4.1: System Architecture of BasoBas

- **Presentation Layer:** This layer involves the BasoBas mobile application which comprises all user interfaces for both tenants and landlords. This layer includes all the user interaction supported in the platform such as authentication and onboarding, searching and filtering of properties, visit request process, rating and review, notifications, and profile and account management.
- **Administration Layer:** The web-based Admin Portal comprises a distinct, web-based platform intended for use by administrators and not by tenants and landlords. It allows for user and listings management, KYC verification, and review moderation. The AI-powered KYC verification makes document verification automated while retaining the option for manual checking when needed.
- **Application and Service Layer:** This layer contains the business logic of BasoBas and uses mainly the services provided by Supabase as the backend. It handles the functionalities of authentication, databases, subscriptions, file storages, and server functions. It also includes third-party services like Clerk to support OTP authentication and Google Maps for location features.
- **Data Layer:** This layer handles application data, storage and management. Data that is structured, such as users, listings, bookings, reviews, and KYC data, is stored in PostgreSQL. Foreign keys and Row-Level Security policies ensure the integrity and access security of that data within the database. On the other hand, the unstructured data, which includes images and videos of properties, users' avatars, and KYC identification documents, is kept in Supabase Storage rather than the relational database since it is designed better for the storage of binary files.

4.1.4 Use Case Diagram

The principal actors are *Tenant* and *Landlord*, with *Admin* managing the platform. Tenant use cases include OTP authentication, onboarding, search and filtering, map-based nearby discovery, saving listings, submitting visit requests, and submitting ratings and reviews. Landlord use cases include property listing creation and management, visit request decision handling, KYC verification, Pro plan subscription and payment, and dashboard management. Admin use cases include user and listing oversight and KYC review.



Figure 4.2: Use Case Diagram for BasoBas

5. Results & Discussion

5.1 Software Development Approach

BasoBas was developed following an **Agile, Iterative-Incremental** software development model. This model was chosen since the system is composed of different functional modules, which can be developed, tested, and improved iteratively while being integrated in a running system. The Iterative-Incremental model differs from the classical Waterfall model in that the Waterfall approach requires one stage to be completed before moving to the next stage, whereas in the former case, development progresses iteratively, each iteration delivering one module of the entire system.

The project lasted for 14 weeks and consisted of ten development phases, which include Requirement Analysis and Literature Review, System Design and Diagrams, React Native (Expo) Configuration and Authentication, Supabase Database Design and Row Level Security, Search and Map-based Feature Discovering, Booking and Review Features, System Implementation and Enhancement, Testing and Debugging, Documentation, and Final Report Writing. Every phase covered certain features of the project while taking into account what was done in previous phases.

This method worked well for BasoBas since there were various relatively independent components like user authentication, sign up process, property listing module, room finding process, visit requests management, KYC verification, ratings and reviews, notifications, and AI recommendations in the system. It was possible to develop each component separately before integrating it into the full application. This meant that at different stages of the development process, the system was already functional, which made it easy to test requirements.

In the entire development process, every module that had been completed was tested before integration with other modules. This helped in identifying and solving issues before any serious problem arose in future during the process of integration. Moreover, the iterative process of the model made it easier for the team to modify the features and make better design decisions through the process of testing.

5.2 Testing, Evaluation and Validation

The completed system was evaluated through five levels of testing: unit testing of the core business logic, integration testing between connected modules, end-to-end system journeys

on a physical device, evaluation of the AI-assisted KYC verification pipeline, and security testing of privacy and access controls. Every test referred to in this section is documented as a formal test case stating its preconditions, execution steps, test data, and expected result, so that each execution can be reproduced and audited. Test outcomes are recorded as *Pass*, *Fail*, *Blocked*, or *Retest*. In addition to manual execution, the unit-level checks are maintained as repeatable automated tests within the project repository, allowing the same verifications to be re-run whenever the codebase changes.

5.2.1 Test Environment

All test executions were performed in the environment summarized in Table 5.1.

Table 5.1: Test Environment

Item	Detail
Backend	Supabase (cloud-hosted PostgreSQL)
Authentication	Clerk (phone number OTP)
Payment gateway	eSewa v2 test (sandbox) environment
Test devices	POCO M2 (Android 12)
Testing period	[Start date] – [End date]

5.2.2 Unit Test Cases

Unit tests exercise the core logic of the application in isolation from the user interface. Each test supplies controlled input to a single logical operation and compares the output against the expected value. Five unit test cases were defined, covering phone number validation, name matching, risk scoring, and property status derivation.

Table 5.2: Unit Test Case UT-01 – Normalization of a Valid Mobile Number

Test ID / Title	UT-01 – A standard ten-digit Nepali mobile number is normalized correctly
Module	Utility service – phone number formatting
Priority	High
Preconditions	Formatting utility available; no network connection required
Test Steps	1. Submit a standard ten-digit mobile number to the normalization routine. 2. Inspect the returned value.
Test Data	“9812345678”
Expected Result	The number is returned in standard international (E.164) format with the +977 country code prefix.
Actual Result	Returned as +9779812345678
Status	Pass

Table 5.3: Unit Test Case UT-02 – Rejection of Malformed Mobile Numbers

Test ID / Title	UT-02 – Malformed phone inputs are rejected safely
Module	Utility service – phone number formatting
Priority	High
Preconditions	Same as UT-01
Test Steps	1. Submit an incomplete number to the normalization routine. 2. Repeat with a non-numeric string, an empty string, and an eleven-digit (over-length) value to cover boundary conditions. 3. Inspect the returned values.
Test Data	Incomplete number (primary case), empty string, alphabetic string, and an over-length number
Expected Result	Every invalid input is rejected gracefully with no value returned; no error or crash occurs.
Actual Result	All invalid inputs were rejected without errors
Status	Pass

Table 5.4: Unit Test Case UT-03 – Name Comparison for Identical Names

Test ID / Title	UT-03 – Identical names receive a perfect similarity score
Module	KYC – identity cross-check between document and profile names
Priority	High
Preconditions	Comparison routine available and confirmed to behave deterministically
Test Steps	1. Compare two identical names that differ only in letter casing and spacing. 2. Inspect the returned similarity score.
Test Data	Two identical names differing only in case and whitespace
Expected Result	Similarity score of 100 (case-insensitive, whitespace-tolerant exact match).
Actual Result	Score of 100
Status	Pass

Table 5.5: Unit Test Case UT-04 – Composite Risk Score Calculation

Test ID / Title	UT-04 – Weighted risk score is computed correctly from component scores
Module	KYC – risk scoring engine
Priority	Critical
Preconditions	Weight configuration loaded; all component scores within the 0–100 range
Test Steps	1. Supply four component scores representing document quality, tamper likelihood, identity consistency, and face match. 2. Compare the composite output with a hand-calculated weighted value.
Test Data	Quality = 90, tamper = 95, identity = 88, face = 91
Expected Result	Composite score ≈ 91 , within the rounding tolerance implied by the configured weights.
Actual Result	≈ 91
Status	Pass

Table 5.6: Unit Test Case UT-05 – Property Status Derivation Threshold

Test ID / Title	UT-05 – Property is flagged as high demand at three or more active visits
Module	Properties – automatic status derivation from open visit count
Priority	Medium
Preconditions	Status derivation logic callable with a synthetic open-visit count
Test Steps	1. Invoke status derivation with two active visits and inspect the resulting status. 2. Repeat with three active visits and inspect the result.
Test Data	Active (open) visit counts of 2 and then 3
Expected Result	Two visits → Available; three visits → High Demand.
Actual Result	Available at two visits; High Demand at three visits
Status	Pass

5.2.3 Integration Test Cases

Integration tests verify that information produced by one module is consumed correctly by another, covering the paths between the mobile application, the cloud database, the identity provider, and the payment gateway. Five integration test cases were executed.

Table 5.7: Integration Test Case IT-01 – Sign-In Creates a Profile Record

Test ID / Title	IT-01 – Successful OTP sign-in persists an authenticated profile
Modules	Authentication (identity provider) ↔ onboarding service ↔ profile records
Priority	Critical
Preconditions	Phone number not previously registered; SMS delivery configured
Test Steps	1. Launch the application and enter a fresh phone number. 2. Receive and enter the correct OTP code. 3. Complete role selection and basic onboarding. 4. Inspect the stored profile records.
Test Data	Fresh test phone number; role selected: Tenant
Expected Result	Sign-in succeeds without error; a new profile record exists with the correct user identifier, phone number, and selected role; repeated sign-ins do not create duplicate records.
Actual Result	Record created with correct identifier, phone, and role; no duplicates
Status	Pass

Table 5.8: Integration Test Case IT-02 – Landlord Onboarding with KYC Submission

Test ID / Title	IT-02 – KYC submission populates all related records and sets review status
Modules	Onboarding wizard ↔ KYC service ↔ document storage ↔ verification records
Priority	Critical
Preconditions	Logged-in landlord account without existing KYC; valid sample document images available
Test Steps	1. Start landlord onboarding and complete personal details. 2. Upload citizenship and electricity bill images in the KYC step. 3. Submit the application. 4. Inspect stored files, verification records, and review status.
Test Data	Two clear sample document images; consistent name across all fields
Expected Result	Images are uploaded to private storage with their locations recorded in the database; all related verification records are created and linked to the submitting user; verification status is set to Under Review.
Actual Result	Records populated correctly; status set to Under Review
Status	Pass

Table 5.9: Integration Test Case IT-03 – Listing Publication with Photos

Test ID / Title	IT-03 – Listing creation persists property details and ordered photos
Modules	Listing wizard ↔ property service ↔ image storage ↔ property records
Priority	Critical
Preconditions	Landlord verification status is Verified (publishing is gated on verification)
Test Steps	1. Complete all listing wizard steps (details, price, amenities, photos). 2. Upload three photos and reorder them by moving the third photo to the first position. 3. Publish the listing. 4. Inspect the stored property record, photo order, and search visibility.
Test Data	Three JPG photos of approximately 2 MB each; monthly rent Rs. 15,000
Expected Result	The property record is created with all entered values; photo locations are persisted in the chosen display order; the listing appears in tenant search immediately after publication.
Actual Result	Record created; photo order preserved; listing visible in search
Status	Pass

Table 5.10: Integration Test Case IT-04 – Visit Acceptance Unlocks Address Visibility

Test ID / Title	IT-04 – Landlord approval reveals the exact address to the requesting tenant only
Modules	Request management ↔ visit service ↔ database access policies ↔ tenant property screen
Priority	Critical
Preconditions	One property with a hidden exact address; one pending tenant visit request
Test Steps	1. As a tenant with a pending request, confirm the exact address is not visible. 2. As the landlord, accept the visit request. 3. As the same tenant, reopen the property and request screens. 4. Confirm that a different tenant’s pending request still shows no address.
Test Data	One approved tenant and one pending tenant on the same property
Expected Result	The approved tenant receives the full address and co-ordinates, while the pending tenant continues to receive redacted location information. The restriction is enforced at the database access layer, not merely in the interface.
Actual Result	Address unlocked for the approved tenant only
Status	Pass

Table 5.11: Integration Test Case IT-05 – eSewa Payment Activates Pro Plan

Test ID / Title	IT-05 – A successful sandbox payment activates the subscription server-side
Modules	Checkout ↔ order creation ↔ eSewa sandbox ↔ payment verification ↔ subscription records
Priority	Critical
Preconditions	eSewa test credentials configured; verification functions deployed
Test Steps	1. Select the 3-Month Pass and tap Subscribe. 2. Confirm that the payment request carries the correct signed amount and transaction reference, sourced from the server-side plan configuration rather than the client. 3. Complete payment on the eSewa test page using test credentials. 4. Return to the application and observe the success screen. 5. Check the subscription state in the profile and inspect the stored subscription record.
Test Data	Plan: 3-Month Pass; eSewa test credentials
Expected Result	The backend independently verifies the payment response before activation; the Pro badge and subscription card appear on the profile; the subscription record is created with Active status and the correct expiry date (+3 months); activation completes within a few seconds of returning to the app under simulated test conditions.
Actual Result	Pro activated after server-side verification; record Active
Status	Pass

5.2.4 System Test Cases (End-to-End Journeys)

Full user journeys were walked through on a physical device without any intervention. Each journey chains several of the integration flows described above and observes the outcome purely through the user interface.

Table 5.12: System Journey SYS-J1 – Tenant Registration to Visit Request

#	Step	Expected Result
1	Enter phone number, receive OTP, and enter the code	OTP accepted; routed to onboarding
2	Select Tenant role and complete onboarding	Lands on the home screen with listings
3	Open a listing and tap Request Visit	Confirmation shown; visit created as Pending
4	Open the My Visits tab	Visit listed under Upcoming with a Pending badge
Result: Completed in approximately two minutes under simulated test conditions — Pass		

Table 5.13: System Journey SYS-J2 – Landlord Listing to Rental Finalization

#	Step	Expected Result
1	Attempt to publish before completing KYC	Publish blocked; redirected to mandatory KYC
2	Submit KYC and await verification	Verification timeline shows Verified
3	Publish the listing	Listing live and discoverable in tenant search
4	Accept an incoming visit request	Tenant’s screen unlocks the exact address
5	Mark the request Rental Finalized	Property becomes Occupied; other requests closed automatically
Result: Completed successfully — Pass		

Table 5.14: System Journey SYS-J3 – eSewa Subscription Payment

#	Step	Expected Result
1	Open Plans, select the 3-Month Pass, and subscribe	Redirected to the eSewa test page showing the correct amount
2	Pay with test credentials	Redirect back to the app; backend verifies the payment
3	Open the profile	Pro badge and subscription card with expiry date visible
Result: Passed; activation completed in approximately five seconds (simulated) — Pass		

5.2.5 KYC Verification Evaluation

The KYC verification module automates document checking by combining several external AI services, each handling one stage of the process: text extraction from submitted documents, tamper and forgery detection, identity consistency checking between extracted details and user-entered information, and face comparison between the document photo and the user’s selfie. Since no single service covers the entire workflow, the pipeline chains these services together, converts their outputs into four component scores (document quality, tamper likelihood, identity consistency, and face match), aggregates them into a composite risk score, and routes uncertain cases to manual administrative review.

The pipeline was exercised against a labelled dataset of 25 documents seeded through the admin test facility. Each document was prepared to match one of four categories with a known expected outcome, as shown in Table 5.15.

Table 5.15: KYC Evaluation Dataset Composition

Document Category (Preset)	Count	Expected Outcome (Label)
Valid and clear	10	Auto-approved
Tampered	6	Auto-rejected
Blurry / low quality	5	Flagged for manual review
Name mismatch	4	Flagged for review or rejected

For every seeded document, one evaluation record was produced containing the category preset, the ground-truth label, the pipeline outcome, and the four component scores (document quality, tamper detection, identity consistency, and face match) together with the composite risk score. An evaluation was marked successful when the pipeline outcome matched the ground-truth label.

From the sixteen genuine-or-tampered documents (ten clear plus six tampered), a confusion matrix for tamper detection was constructed, as shown in Table 5.16.

Table 5.16: Confusion Matrix for Tamper Detection

	Predicted Genuine	Predicted Tampered
Actual Genuine	TP = [x]	FN = [x]
Actual Tampered	FP = [x]	TN = [x]

Precision, recall, and F1-score were derived from the matrix as follows:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F1 = \frac{2 \times P \times R}{P + R}$$

Overall agreement between pipeline outcomes and ground-truth labels across all 25 documents was computed as:

$$\text{Accuracy} = \frac{\text{matched outcomes}}{25} \times 100\% = [\text{XX}]\%$$

It should be noted that the KYC module is not fully autonomous at this stage. Because each stage depends on an external service, the overall accuracy is bounded by the weakest stage, and results vary with document type, image quality, and lighting conditions. The evaluation therefore serves as a baseline measurement of the pipeline’s current behaviour rather than proof of production readiness. The human-in-the-loop review step remains essential for flagged cases, and since every stage operates independently behind a common interface, any individual external service can be replaced with a better-performing alternative without affecting the rest of the workflow.

5.2.6 Security Test Cases

Security testing focused on the platform’s trust guarantees: location privacy and data access control. The tests were executed directly against the data layer using database tooling and API testing tools, confirming that protections hold even when requests bypass the mobile interface.

Table 5.17: Security Test Case ST-01 – Location Hidden Without Approved Visit

Objective	Confirm that the exact address and coordinates of a property are withheld from tenants without an approved visit
Method	Query the property record directly as a tenant who holds no approved visit on that property, and capture the returned fields
Expected Result	Exact address, latitude, and longitude are absent from the response; neighbourhood-level information may remain visible
Actual Result	Private fields were not returned
Status	Pass

Table 5.18: Security Test Case ST-02 – Direct Access to Another User’s Document

Objective	Verify that KYC documents cannot be retrieved by users other than the owner or an administrator
Method	Copy the storage location of one user’s KYC document and request that object directly from a different user’s session
Expected Result	Access is forbidden; storage policies restrict reads to the owner and administrators
Actual Result	Access denied (403 Forbidden)
Status	Pass

Table 5.19: Security Test Case ST-03 – Cross-User Modification Blocked

Objective	Confirm that a user cannot modify another user’s profile
Method	From one account, attempt to update the profile record belonging to a different account
Expected Result	No rows are affected; row-level access rules restrict each account to its own records, silently blocking the write
Actual Result	Zero rows affected
Status	Pass

Table 5.20: Security Test Case ST-04 – OTP Brute-Force Lockout

Objective	Verify that repeated incorrect OTP attempts trigger a temporary lockout
Method	Enter an incorrect OTP code three consecutive times during sign-in and observe the error messaging and retry behaviour
Expected Result	After the third failure, further attempts are rejected and the account remains locked for approximately five minutes with a countdown message
Actual Result	Account locked for five minutes
Status	Pass

5.2.7 Traceability Summary

Table 5.21 maps every executed test suite to its coverage and outcome, demonstrating that all specified functionality was exercised during validation.

Table 5.21: Test Suite Traceability Summary

Suite	Test IDs	Total	Pass Rate
Unit	UT-01 – UT-05	5	100%
Integration	IT-01 – IT-05	5	100%
System journeys	SYS-J1 – J3	3	100%
KYC evaluation	EV-01 – EV-27	25 documents + metrics	[XX]%
Security	ST-01 – ST-04	4	100%

Overall, the unit, integration, system-journey, and security suites passed completely, confirming that the implemented modules behave according to their specifications. The KYC evaluation documents the present behaviour of the automated verification pipeline, whose stages rely on external services and whose flagged outcomes continue to be confirmed through manual review before a final decision is issued.

6. Conclusions

The conclusion part summarizes the whole report by highlighting all the chapters and their significance and the importance of the project and the achievements.

7. Limitations and Future enhancement

This chapter should contain major limitations of the project and the further enhancement of the project soon with a different but related approach. **Referencing checking here**[19]

References

- [1] Central Bureau of Statistics Nepal. National population and housing census 2021: National report, 2023. Accessed: 2026-03-03.
- [2] S. Indrayani and S. Istikomah. Integrated web-based platform for boarding house rental services: Information architecture and booking workflow design. *Bit-Tech (Binary Digital Technology)*, 8(3):3834–3845, April 2026.
- [3] S. S. Diaz, J. R. M. C. Layos, C. A. O. Pagaduan, R. L. G. Sarmiento, A. G. T. Sison, and E. A. Serrano. Roommeet: A renting mobile application. In *Proceedings of the 2024 9th International Conference on Intelligent Information Technology (ICIIT)*, pages 564–571, Ho Chi Minh City, Vietnam, February 2024.
- [4] A. Farooq, I. Hussain, H. Iman, N. Kiran, N. I. Kajla, and M. A. Jamil. Roommate: An ai optimized space rental system. *Journal of Statistical Computing and Interdisciplinary Research*, 7(1):51–65, 2025.
- [5] C. Hess, A. Acolin, R. Walter, I. Kennedy, S. Chasins, and K. Crowder. Searching for housing in the digital age: Neighborhood representation on internet rental housing platforms across space, platform, and metropolitan segregation. *Environment and Planning A: Economy and Space*, pages 1–21, 2021.
- [6] I. Zaman, M. M. Khan, T. Rahman, and M. S. Rahman. An interactive web platform and android application based on room sharing service for students. In *Proceedings of the 2021 IEEE World AI IoT Congress (AIIoT)*, pages 0127–0131, 2021.
- [7] G. Tyagi, T. Kumar, T. Tyagi, R. Kumar, R. Maheshwari, and R. Kaushik. Leveraging conversational ai in an intelligent room rental platform. *Computer Research and Development*, 25(5):247–256, 2025.
- [8] V. Balpande, A. Kumre, B. Dongarwar, H. Parate, T. Selukar, and P. Ganvir. Implementation & development of an application for room searching. *International Journal for Research in Applied Science & Engineering Technology (IJRASET)*, 12(4), April 2024.

- [9] A. Wakodikar, O. Thakre, and R. Yadav. Design and implementation of a to-let: A web based room finder application. *International Journal of Advanced Research in Science, Communication and Technology (IJARSCT)*, 6(6), March 2026.
- [10] Fensen, E. Dewayani, and Wasino. Website application for locating and reserving house and boardroom for rent based on selected location or region. *IOP Conference Series: Materials Science and Engineering*, 1007:012112, 2020.
- [11] O. J. Uchechukwu and A. A. Shehu. A potential online platform that aims to make renting and buying properties easy and hassle-free for foreigners in russia. *ScienceOpen Preprints*, May 2025.
- [12] Jean-Charles Rochet and Jean Tirole. Platform competition in two-sided markets. *Journal of the European Economic Association*, 1(4):990–1029, 2003.
- [13] E. F. Codd. A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387, 1970.
- [14] Roger W. Sinnott. Virtues of the haversine. *Sky and Telescope*, 68(2):159, 1984.
- [15] Audun Jøsang, Roslan Ismail, and Colin Boyd. A survey of trust and reputation systems for online service provision. *Decision Support Systems*, 43(2):618–644, 2007.
- [16] Steven Tadelis. Reputation and feedback systems in online platform markets. *Annual Review of Economics*, 8(1):321–340, 2016.
- [17] Nicole Ozer. Location-based services: Time for a privacy check-in. ACLU of Northern California Issue Paper, 2010. Accessed: 2026-03-03.
- [18] Teresa Scassa and Anca Sattler. Location-based services and privacy. *Canadian Journal of Law and Technology*, 9(1), 2011.
- [19] AAAAA BBBB and XXXX YYYY. Nnn nnn nnnn nnnn nnnnn. *International Journal of Artificial Intelligence & Applications*, 10(4):47–55, 2019.

Appendices

It should contain the information that is somehow missing in the main body and it presents extra information about your project like program source code, raw data, and so on.